

Contents

Abstract	2
Introduction	3
Literature Review	4
Methodology	5
Results	6
Report	7
Discussion	8
Conclusion	9
References	10
Appendices	11
Appendix A: API Endpoints Summary	11
Appendix B: Sample Request and Response	12
Appendix C: Sample Code Snippet	13
Appendix D: System Screenshots	14
GET /books response	14
POST /borrow execution	16
GET /books after borrowing	18
POST /return execution	20
GET /users/{id}/overdue	22
PUT /books/{id}/ update book	24
GET /books after Update	25
PATCH /books/{id}/ partial update	26
GET /books after partial Update (PATCH)	28
DELETE /books/{id}/ Delete Book	29
GET /books after delete (DELETE BOOK)	30
Appendix E: Tools and Technologies Used	31
Appendix F: GitHub Repository	32
API DESIGN (SYSTEM ARCHITECTURE DIAGRAM)	33
Appendix G: Assumptions	34

Abstract

This project presents the design and implementation of a RESTful Application Programming Interface (API) for a digital library system tailored to support academic environments. The primary objective of the system is to improve access to library resources by enabling efficient searching, borrowing, and returning of books through a structured and automated interface. The API was developed using Python with the FastAPI framework, which facilitates high performance and asynchronous processing, allowing multiple users to interact with the system simultaneously.

The system provides key functionalities such as searching for books by title, author, or category, managing borrowing and returning processes, and tracking overdue books with calculated fines. By adopting RESTful principles, the API ensures stateless communication, clear resource-based endpoints, and standardized data exchange using JSON. This contributes to a scalable and maintainable architecture suitable for real-world applications.

This project aligns with Sustainable Development Goal 4 (Quality Education), as it promotes improved access to learning materials through digital transformation. By streamlining library operations, the system reduces manual workload, enhances accuracy, and enables faster access to educational resources for students and staff.

Despite its effectiveness, the project faced challenges such as handling concurrent requests using asynchronous programming and ensuring proper validation of user inputs. These challenges were addressed through careful implementation of async/await structures and structured data models. Overall, the developed API demonstrates how modern web technologies can be leveraged to enhance library services and support educational development in a digital era.

Introduction

Libraries play a fundamental role in supporting education by providing access to books, research materials, and other learning resources. However, many traditional library systems still rely on manual processes for managing book inventories, borrowing, and record-keeping. These manual methods are often time-consuming, prone to human error, and inefficient when handling multiple users simultaneously. With the rapid advancement of digital technologies, there is a growing need to modernize library systems to improve accessibility, efficiency, and overall service delivery.

This project focuses on the development of a digital library system using a RESTful Application Programming Interface (API). The system is designed to allow users to search for books, borrow and return them, and monitor overdue items through a structured and automated platform. By implementing RESTful principles, the system uses standard HTTP methods such as GET, POST, PUT, and DELETE to manage resources effectively. The API is built using the FastAPI framework in Python, which supports asynchronous operations, enabling multiple users to interact with the system concurrently without performance degradation.

The motivation behind this project is to enhance the efficiency of library operations while improving user experience. Students and staff can easily access information about available books, perform transactions quickly, and receive accurate updates on borrowed materials. This reduces the workload on library staff and minimizes delays associated with manual record management.

Furthermore, the project is aligned with Sustainable Development Goal 4 (Quality Education), which emphasizes inclusive and equitable access to education. By digitizing library services, the system promotes easier access to learning materials, thereby supporting academic success and continuous learning. In this context, the developed API serves as a practical solution that demonstrates how modern software technologies can be applied to improve educational infrastructure.

Overall, this project highlights the importance of integrating RESTful APIs into library management systems and showcases how such technologies can contribute to more efficient, scalable, and user-friendly services in educational institutions.

Literature Review

The adoption of digital technologies in library management has been widely discussed in recent years, particularly with the rise of web-based systems and Application Programming Interfaces (APIs). Traditional library systems, which rely heavily on manual processes, have been shown to be inefficient in handling large volumes of data and multiple users. According to studies on digital transformation in education, automated library systems significantly improve resource accessibility, reduce administrative workload, and enhance user satisfaction.

RESTful APIs have become a standard approach in modern system design due to their simplicity, scalability, and interoperability. Representational State Transfer (REST) is an architectural style that uses standard HTTP methods such as GET, POST, PUT, and DELETE to manage resources. Research indicates that RESTful APIs are particularly effective in distributed systems because they enable seamless communication between clients and servers while maintaining stateless interactions. This makes them suitable for applications like library systems where multiple users may access resources concurrently.

FastAPI, a modern Python web framework, has gained popularity for building APIs due to its high performance and support for asynchronous programming. Studies comparing web frameworks highlight FastAPI's ability to handle concurrent requests efficiently, making it ideal for systems requiring real-time interaction. Its use of automatic data validation through Pydantic models also improves reliability and reduces the likelihood of errors in data handling.

In the context of education, digital library systems play a crucial role in supporting Sustainable Development Goal 4 (Quality Education). Literature shows that access to digital resources enhances learning opportunities, especially in institutions with limited physical resources. By integrating APIs into library systems, institutions can provide faster and more reliable access to educational materials, thereby improving learning outcomes.

However, challenges such as data security, system scalability, and user authentication are commonly identified in existing studies. Many basic implementations do not include advanced features like secure access control or persistent data storage, which are necessary for real-world deployment. Despite these limitations, RESTful API-based systems provide a strong foundation for developing scalable and efficient library management solutions.

Overall, the literature supports the use of RESTful APIs and modern frameworks like FastAPI in transforming traditional library systems into efficient digital platforms that enhance accessibility, usability, and performance.

Methodology

This project adopted a practical and iterative development approach to design and implement a RESTful API for a digital library system. The methodology focused on identifying system requirements, designing API endpoints, implementing functionality using a suitable framework, and testing the system to ensure correctness and efficiency. The overall goal was to create a simple, scalable, and user-friendly solution that aligns with real-world library operations.

The first phase involved requirement analysis, where the core functionalities of the system were identified based on the assignment brief. These included searching for books, borrowing and returning books, tracking overdue items, and supporting multiple users. Additional enhancements such as updating and deleting book records were also considered to extend the system's capabilities. These requirements guided the design of the API structure and endpoints.

In the design phase, the system was structured following RESTful architectural principles. Resources such as books and users were defined, and appropriate HTTP methods were assigned to each operation. For example, GET was used to retrieve book data, POST for borrowing and returning operations, and PUT, PATCH, and DELETE for managing book records. The API endpoints were designed to be clear, consistent, and easy to understand, ensuring efficient interaction between the client and the server.

The implementation phase was carried out using Python with the FastAPI framework. FastAPI was selected due to its high performance, simplicity, and built-in support for asynchronous programming. The use of `async` and `await` enabled the system to simulate multiple users interacting with the API simultaneously. Data models were defined using Pydantic to ensure proper validation of inputs and structured data handling. The system used in-memory data structures, such as lists and dictionaries, to store information about books, users, and borrowing records.

Testing was conducted using the built-in Swagger interface provided by FastAPI. Each endpoint was tested individually to verify correct functionality, including successful responses and proper error handling for invalid requests. Scenarios such as borrowing unavailable books and returning non-existent records were also tested to ensure robustness.

Overall, the methodology ensured a systematic development process, resulting in a functional and efficient RESTful API that meets the requirements of a digital library system while demonstrating key concepts of modern API design and implementation.

Results

The implementation of the RESTful API for the digital library system produced a fully functional and responsive application capable of handling core library operations efficiently. The system was successfully developed using Python and the FastAPI framework, and all defined endpoints were tested to ensure proper functionality. The results demonstrate that the API meets the requirements outlined in the project brief and performs reliably under simulated multi-user conditions.

The GET /books endpoint successfully retrieves a list of available books and supports filtering by title, author, and category. This functionality allows users to quickly locate specific resources without manually searching through large datasets. The POST /borrow endpoint enables users to borrow books, automatically updating the availability status of the selected book and generating a due date. Similarly, the POST /return endpoint processes book returns and restores the availability status, ensuring accurate tracking of library inventory.

The system also effectively tracks overdue books through the GET /users/{user_id}/overdue endpoint. This feature calculates the number of days a book is overdue and determines the corresponding fine based on a predefined rate. The results confirm that the system correctly identifies overdue records and provides structured output in JSON format, making it easy for users and administrators to interpret.

Additional endpoints such as PUT, PATCH, and DELETE were implemented to manage book records, allowing full and partial updates as well as removal of entries. These operations demonstrate the flexibility and completeness of the RESTful API design. Furthermore, the use of asynchronous programming enabled the system to simulate concurrent user interactions without performance issues, highlighting its scalability potential.

Overall, the results indicate that the developed API is efficient, reliable, and capable of supporting essential library functions. The successful execution of all endpoints, along with proper error handling and data validation, confirms that the system is well-structured and suitable for further development into a fully deployed application.

Report

This report presents the development and evaluation of a RESTful API designed to support a digital library system. The primary objective of the project was to create a simple, efficient, and scalable solution that enables users to search for books, borrow and return them, and monitor overdue items. The system was implemented using Python with the FastAPI framework, which provided the necessary tools for building high-performance APIs with asynchronous capabilities.

The development process began with identifying the key requirements of the system, which included managing book records, supporting multiple users, and ensuring accurate tracking of borrowing activities. Based on these requirements, a RESTful architecture was adopted, where resources such as books and users were represented through clearly defined endpoints. Standard HTTP methods were used to perform operations, ensuring consistency and ease of use.

The system was structured to include several endpoints, each serving a specific purpose. The GET /books endpoint allows users to retrieve and filter book information, while the POST /borrow and POST /return endpoints handle borrowing and returning transactions. The GET /users/{user_id}/overdue endpoint provides information on overdue books and associated fines. Additional endpoints such as PUT, PATCH, and DELETE were implemented to enhance the management of book records, demonstrating a complete CRUD (Create, Read, Update, Delete) functionality.

Throughout the development process, emphasis was placed on ensuring code quality, proper data validation, and efficient handling of multiple requests. The use of asynchronous programming enabled the system to simulate concurrent access, reflecting real-world usage scenarios. Testing was carried out using the FastAPI Swagger interface, where each endpoint was verified for correctness and reliability.

The findings from this project indicate that RESTful APIs are highly effective in modernizing traditional systems such as libraries. The implemented solution not only meets the functional requirements but also provides a strong foundation for future enhancements, including database integration and user authentication. Overall, the project demonstrates the practical application of RESTful principles in building efficient and user-friendly digital systems.

Discussion

The development of the RESTful API for the digital library system demonstrates the effectiveness of modern web technologies in improving traditional service delivery. The system successfully addresses key challenges associated with manual library operations, such as inefficiency, limited accessibility, and difficulty in managing multiple users. By implementing a structured API, the project provides a scalable solution that enables seamless interaction between users and the system.

One of the major strengths of the system is its adherence to RESTful principles, including stateless communication, resource-based endpoints, and the use of standard HTTP methods. These principles contribute to a clean and maintainable architecture, making the system easier to extend and integrate with other applications in the future. Additionally, the use of FastAPI enhances performance through asynchronous programming, allowing the system to handle multiple requests simultaneously without significant delays.

The results also highlight the importance of data validation and error handling in ensuring system reliability. By implementing checks for user existence, book availability, and invalid requests, the system minimizes the risk of inconsistencies and enhances user experience. Furthermore, the ability to track overdue books and calculate fines adds practical value, as it reflects real-world library operations and promotes accountability among users.

However, the system has some limitations that should be acknowledged. The use of in-memory data storage means that data is not persistent and will be lost when the server is restarted. This limits the system's applicability in real-world environments where long-term data storage is required. Additionally, the absence of authentication and authorization mechanisms poses potential security risks, as any user can access and modify data without restrictions.

To address these limitations, future improvements could include integrating a database management system such as MySQL or PostgreSQL for persistent storage, as well as implementing user authentication to enhance security. Expanding the system to include features such as reservation of books and fine payment processing could further improve its functionality.

In conclusion, the project demonstrates that RESTful APIs provide a robust foundation for developing efficient digital library systems. While the current implementation meets the basic requirements, it also offers opportunities for further enhancement to support more advanced and secure real-world applications.

Conclusion

This project successfully demonstrated the design and implementation of a RESTful API for a digital library system using Python and the FastAPI framework. The primary objective was to develop a simple, efficient, and scalable solution that supports key library operations such as searching for books, borrowing and returning materials, and tracking overdue items. The results confirm that the system meets these objectives and provides a functional platform that improves the management of library resources.

Through the application of RESTful principles, the system was structured using clear and consistent endpoints, standard HTTP methods, and stateless communication. This approach ensured that the API is easy to understand, maintain, and extend. The use of asynchronous programming further enhanced the system's ability to handle multiple user requests simultaneously, reflecting real-world usage scenarios. Additionally, proper data validation and error handling contributed to the reliability and robustness of the application.

The project also highlighted the importance of digital solutions in supporting Sustainable Development Goal 4 (Quality Education). By improving access to library resources and streamlining operations, the system contributes to a more efficient educational environment where students and staff can easily obtain the materials they need for learning and research.

Despite its success, the system has certain limitations, including the use of in-memory data storage and the absence of advanced security features such as user authentication. These limitations provide opportunities for future enhancements, such as integrating a database for persistent storage and implementing secure access controls.

In conclusion, the developed RESTful API serves as a strong foundation for modernizing traditional library systems. It demonstrates how contemporary web technologies can be applied to improve efficiency, accessibility, and overall service delivery in educational institutions. With further development, the system has the potential to evolve into a fully functional and scalable solution suitable for real-world deployment.

References

- Fielding, R. T. (2000). Architectural Styles and the Design of Network-Based Software Architectures (Doctoral dissertation, University of California, Irvine).
- Richardson, L., & Ruby, S. (2007). RESTful Web Services. O'Reilly Media.
- OpenAPI Initiative. (2024). OpenAPI Specification. Retrieved from <https://www.openapis.org>
- FastAPI Documentation. (2025). FastAPI – Modern, fast (high-performance) web framework for building APIs with Python. Retrieved from <https://fastapi.tiangolo.com>
- Python Software Foundation. (2025). Python Documentation. Retrieved from <https://docs.python.org>
- Pydantic Documentation. (2025). Data validation and settings management using Python type annotations. Retrieved from <https://docs.pydantic.dev>
- World Bank. (2023). Digital Transformation in Education. Washington, DC: World Bank Publications.
- United Nations. (2015). Transforming our world: The 2030 Agenda for Sustainable Development. Retrieved from <https://sdgs.un.org/goals/goal4>
- Kumar, V., & Gupta, P. (2021). "Digital Library Management Systems: A Review of Trends and Technologies." International Journal of Information Systems, 13(2), 45–58.
- Almeida, F., & Monteiro, J. (2020). "The Role of APIs in Modern Software Development." Journal of Software Engineering and Applications, 13(5), 123–135.

Appendices

Appendix A: API Endpoints Summary

Method	Endpoint	Description	
GET	/books	Retrieve all books (with optional filters)	
GET	/users	Retrieve all users	
POST	/borrow	Borrow a book	
POST	/return	Return a borrowed book	
GET	/users/{user_id}/overdue	View overdue books and fines	
PUT	/books/{id}	Update full book details	
PATCH	/books/{id}	Partially update book	
DELETE	/books/{id}	Delete a book	

Appendix B: Sample Request and Response

Borrow Book Request

```
{  
  "user_id": 1,  
  "book_id": 1  
}
```

Response

```
{  
  "message": "Book borrowed successfully",  
  "due_date": "2026-05-02"  
}
```

Appendix C: Sample Code Snippet

```
@app.post("/borrow")

async def borrow_book(request: BorrowRequest):

    book = next((b for b in books if b["id"] == request.book_id), None)

    if not book:

        raise HTTPException(status_code=404, detail="Book not found")

    if not book["available"]:

        raise HTTPException(status_code=400, detail="Book not available")

    book["available"] = False

    return {"message": "Book borrowed successfully"}
```

Appendix D: System Screenshots

GET /books response

The first screenshot shows the REST client interface for the GET /books endpoint. The parameters tab is active, displaying three query parameters: title, author, and category. Each parameter has a text input field. The title field contains the text 'title', the author field contains 'author', and the category field contains 'category'. The data types for each parameter are listed as 'string | (string | null) (query)'. At the bottom of the parameters section are 'Execute' and 'Clear' buttons. A 'Cancel' button is located in the top right corner of the parameters section.

The second screenshot shows the 'Responses' tab of the REST client. It displays the following information:

- Curl:**

```
curl -X 'GET' \
'http://127.0.0.1:8000/books' \
-H 'accept: application/json'
```
- Request URL:** `http://127.0.0.1:8000/books`
- Server response:**
- Code:** 200
- Response body:**

```
{
  "id": 1,
  "title": "Python Basics",
  "author": "Joshua Yaffa",
  "category": "Programming",
  "available": true
},
{
  "id": 2,
  "title": "Advanced Python",
  "author": "Josephine Ashley",
  "category": "Programming",
  "available": true
},
{
  "id": 3,
  "title": "Data Science with Python",
  "author": "John Doe",
  "category": "Data Science",
  "available": false
}
```

127.0.0.1:8000/docs#/default/get_books_books_get

200

Response body

```
[
  {
    "id": 1,
    "title": "Python Basics",
    "author": "Joshua Yaffa",
    "category": "Programming",
    "available": true
  },
  {
    "id": 2,
    "title": "Advanced Python",
    "author": "Josephine Ashley",
    "category": "Programming",
    "available": true
  },
  {
    "id": 3,
    "title": "Data Science Intro",
    "author": "Phinney Ashley",
    "category": "Data",
    "available": true
  },
  {
    "id": 4,
    "title": "Machine Learning",
    "author": "Thomas Ashley",
    "category": "AI",
    "available": true
  }
]
```

Response headers

```
content-length: 2994
content-type: application/json
date: Wed, 29 Apr 2026 10:31:19 GMT
server: uvicorn
```

Responses

Code	Description	Links
------	-------------	-------

127.0.0.1:8000/docs#/default/get_books_books_get

Responses

Code	Description	Links
200	Successful Response	No links
	Media type: application/json	
	Controls Accept header.	
	Example Value Schema	
	<pre>[{ "additionalProp1": {} }]</pre>	
422	Validation Error	No links
	Media type: application/json	
	Example Value Schema	
	<pre>{ "detail": [{ "loc": ["string", 0], "msg": "string", "type": "string", "input": "string" }] }</pre>	

POST /borrow execution

The screenshot displays a web browser window with the URL `127.0.0.1:8000/docs#/default/borrow_book_borrow_post`. The browser's address bar and tabs are visible at the top. The main content area shows a REST client interface for the `POST /borrow` endpoint.

Parameters: No parameters are defined for this endpoint.

Request body: The request body is set to `application/json`. The JSON payload is:

```
{  "user_id": 1,  "book_id": 1}
```

Execute: A blue button labeled "Execute" is used to send the request.

Responses: The response section shows the following details:

- Curl:**

```
curl -X 'POST' \  'http://127.0.0.1:8000/borrow' \  -H 'accept: application/json' \  -H 'Content-Type: application/json' \  -d '{  "user_id": 1,  "book_id": 1  }'
```
- Request URL:** `http://127.0.0.1:8000/borrow`
- Server response:**
 - Code:** 200
 - Response body:**

```
{  "message": "Book borrowed successfully",  "due_date": "2026-05-06"}
```
 - Response headers:** `content-length: 64`

127.0.0.1:8000/docs#/default/borrow_book_borrow_post

Response headers

```

content-length: 64
content-type: application/json
date: Wed, 29 Apr 2026 10:36:29 GMT
server: uvicorn

```

Responses

Code	Description	Links
200	Successful Response	No links
Media type: application/json Controls Accept header. Example Value Schema <pre>"string"</pre>		
422	Validation Error	No links
Media type: application/json Example Value Schema <pre>{ "detail": [{ "loc": [</pre>		

127.0.0.1:8000/docs#/default/borrow_book_borrow_post

422 Validation Error

Media type:

Example Value | Schema

```

{
  "detail": [
    {
      "loc": [
        "string",
        0
      ],
      "msg": "string",
      "type": "string",
      "input": "string",
      "ctx": {}
    }
  ]
}

```

POST /return Return Book

GET /users/{user_id}/overdue Get Overdue Books

PUT /books/{book_id} Update Book

PATCH /books/{book_id} Patch Book

GET /books after borrowing

127.0.0.1:8000/docs#/default/get_books_books_get

GET /books Get Books

Parameters

Cancel

Name	Description
title string (string null) (query)	Python Basics
author string (string null) (query)	Joshua Yaffa
category string (string null) (query)	Programming

Execute Clear

Responses

127.0.0.1:8000/docs#/default/get_books_books_get

Responses

Curl

```
curl -X 'GET' \
'http://127.0.0.1:8000/books?title=Python%20Basics&author=Joshua%20Yaffa&category=Programming' \
-H 'accept: application/json'
```

Request URL

<http://127.0.0.1:8000/books?title=Python%20Basics&author=Joshua%20Yaffa&category=Programming>

Server response

Code	Details
200	Response body <pre>{ "id": 1, "title": "Python Basics", "author": "Joshua Yaffa", "category": "Programming", "available": false }</pre> Response headers <pre>content-length: 101 content-type: application/json date: Wed, 29 Apr 2026 10:41:32 GMT server: uvicorn</pre>

Responses

127.0.0.1:8000/docs#/default/get_books_books_get

Responses

Code	Description	Links
200	Successful Response	No links
	Media type: application/json Controls Accept header. Example Value Schema	
	<pre>[{ "additionalProp1": {} }]</pre>	
422	Validation Error	No links
	Media type: application/json Example Value Schema	
	<pre>{ "detail": [{ "loc": ["string", 0], "msg": "string", "type": "string", "input": "string", "ctx": {} }] }</pre>	

28° Search 10:42 29/04/2026

127.0.0.1:8000/docs#/default/get_books_books_get

Media type: application/json
Controls Accept header.
Example Value | Schema

```
[
  {
    "additionalProp1": {}
  }
]
```

422 Validation Error No links

Media type: application/json
Example Value | Schema

```
{
  "detail": [
    {
      "loc": [
        "string",
        0
      ],
      "msg": "string",
      "type": "string",
      "input": "string",
      "ctx": {}
    }
  ]
}
```

POST /borrow Borrow Book

28° Search 10:42 29/04/2026

POST /return execution

127.0.0.1:8000/docs#/default/return_book_return_post

POST /return Return Book

Parameters Cancel

No parameters

Request body required application/json

Edit Value | Schema

```
{  "user_id": 0,  "book_id": 0}
```

28° Search 10:44 29/04/2026

127.0.0.1:8000/docs#/default/return_book_return_post

POST /return Return Book

Parameters Cancel Reset

No parameters

Request body required application/json

Edit Value | Schema

```
{  "user_id": 1,  "book_id": 1}
```

28° Search 10:44 29/04/2026

127.0.0.1:8000/docs#/default/return_book_return_post

Execute Clear

Responses

Curl

```
curl -X 'POST' \
  'http://127.0.0.1:8000/return' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "user_id": 1,
    "book_id": 1
  }'
```

Request URL

http://127.0.0.1:8000/return

Server response

Code	Details
200	Response body <pre>{ "message": "Book returned successfully" }</pre> Response headers <pre>content-length: 40</pre>

127.0.0.1:8000/docs#/default/return_book_return_post

content-length: 40
content-type: application/json
date: Wed, 29 Apr 2026 10:44:40 GMT
server: uvicorn

Responses

Code	Description	Links
200	Successful Response Media type: application/json Controls Accept header. Example Value Schema <pre>"string"</pre>	No links
422	Validation Error Media type: application/json Example Value Schema <pre>{ "detail": [{ "loc": ["string", </pre>	No links

API Design for Library | Library API Assignment Solution | Limkokwing Library REST API

127.0.0.1:8000/docs#/default/return_book_return_post

Controls Accept header.

Example Value | Schema

"string"

422 Validation Error No links

Media type
application/json

Example Value | Schema

```
{
  "detail": [
    {
      "loc": [
        "string",
        0
      ],
      "msg": "string",
      "type": "string",
      "input": "string",
      "ctx": {}
    }
  ]
}
```

GET /users/{user_id}/overdue Get Overdue Books

PUT /books/{book_id} Update Book

GET /users/{id}/overdue

127.0.0.1:8000/docs#/default/get_overdue_books_users_user_id_overdue_get

GET /users/{user_id}/overdue Get Overdue Books

Parameters

Try it out

Name	Description
user_id * required integer (path)	user_id

Responses

Code	Description	Links
200	Successful Response	No links

Media type
application/json

Controls Accept header.

Example Value | Schema

"string"

28° Search 10:47 29/04/2026

127.0.0.1:8000/docs#/default/get_overdue_books_users_user_id_overdue_get

GET /users/{user_id}/overdue Get Overdue Books

Parameters

Name	Description
user_id * required integer (path)	1

Execute

Responses

Code	Description	Links
200	Successful Response	No links

Media type: application/json

Controls Accept header.

Example Value | Schema

127.0.0.1:8000/docs#/default/get_overdue_books_users_user_id_overdue_get

Curl

```
curl -X 'GET' \
  'http://127.0.0.1:8000/users/1/overdue' \
  -H 'accept: application/json'
```

Request URL

http://127.0.0.1:8000/users/1/overdue

Server response

Code	Details
200	Response body <pre>{ "overdue_books": [] }</pre> Response headers <pre>content-length: 20 content-type: application/json date: Wed, 29 Apr 2026 10:48:23 GMT server: uvicorn</pre>

Responses

Code	Description	Links
200	Successful Response	No links

PUT /books/{id}/ update book

127.0.0.1:8000/docs/

PUT /books/{book_id} Update Book

Parameters Cancel

Name	Description
book_id * required integer (path)	1

Request body required application/json

Edit Value | Schema

```
{
  "additionalProp1": {}
}
```

127.0.0.1:8000/docs/

PUT /books/{book_id} Update Book

Parameters Cancel Reset

Name	Description
book_id * required integer (path)	1

Request body required application/json

Edit Value | Schema

```
{
  "id": 1,
  "title": "Java Basics",
  "author": "Katibi Yaffa",
  "category": "Programming",
  "available": true
}
```


127.0.0.1:8000/docs/#

Responses

Curl

```
curl -X 'PUT' \
  'http://127.0.0.1:8000/books/1' \
  -H 'accept: application/json' \
  -H 'Content-type: application/json' \
  -d '{
    "id": 1,
    "title": "Java Basics",
    "author": "Katibi Yaffa",
    "category": "Programming",
    "available": true
  }'
```

Request URL

http://127.0.0.1:8000/books/1

Server response

Code	Details
200	Response body <pre>{ "message": "Book updated", "book": { "id": 1, "title": "Java Basics", "author": "Katibi Yaffa", "category": "Programming", "available": true } }</pre> Download

GET /books after Update

127.0.0.1:8000/docs#/default/get_books_books_get

GET /books Get Books

Parameters

Name	Description
title	title
author	author
category	category

Execute Clear

Responses

127.0.0.1:8000/docs#/default/get_books_books_get

Curl

```
curl -X 'GET' \
  'http://127.0.0.1:8000/books' \
  -H 'accept: application/json'
```

Request URL

http://127.0.0.1:8000/books

Server response

Code Details

200

Response body

```
[
  {
    "id": 1,
    "title": "Java Basics",
    "author": "Katibi Yaffa",
    "category": "Programming",
    "available": true
  },
  {
    "id": 2,
    "title": "Advanced Python",
    "author": "Josephine Ashley",
    "category": "Programming",
    "available": true
  },
  {
    "id": 3,
    "title": "Data Science Intro",
    "author": "Phinney Ashley",
    "category": "Data",
    "available": true
  }
]
```

PATCH /books/{id}/ partial update

127.0.0.1:8000/docs#/

PATCH /books/{book_id} Patch Book

Parameters

Cancel Reset

Name	Description
book_id * required	
Integer	
(path)	

Request body required

application/json

Edit Value Schema

```
{
  "id": 2,
  "title": "Advanced SQL",
  "author": "Josephine Ashley",
  "category": "Database",
  "available": true
}
```

127.0.0.1:8000/docs#/ Execute Clear

Responses

Curl

```
curl -X 'PATCH' \
  'http://127.0.0.1:8000/books/2' \
  -H 'accept: application/json' \
  -H 'Content-type: application/json' \
  -d '{
    "id": 2,
    "title": "Advanced SQL",
    "author": "Josephine Ashley",
    "category": "Database",
    "available": true
  }'
```

Request URL

http://127.0.0.1:8000/books/2

Server response

Code	Details
200	Response body <pre>{ "message": "Book patched", "book": { "id": 2, "title": "Advanced SQL", "author": "Josephine Ashley", "category": "Database", "available": true } }</pre>

127.0.0.1:8000/docs#/ Server response

Code	Details
200	Response body <pre>{ "message": "Book patched", "book": { "id": 2, "title": "Advanced SQL", "author": "Josephine Ashley", "category": "Database", "available": true } }</pre> Download Response headers <pre>content-length: 132 content-type: application/json date: Wed, 29 Apr 2026 11:01:40 GMT server: uvicorn</pre>

Responses

Code	Description	Links
200	Successful Response	No links

Media type

application/json

Controls Accept header.

Example Value Schema

"string"

GET /books after partial Update (PATCH)

127.0.0.1:8000/docs#/default/get_books_books_get

GET /books Get Books

Parameters

Cancel

Name	Description
title	title
author	author
category	category

Execute Clear

127.0.0.1:8000/docs#/default/get_books_books_get

Responses

Curl

```
curl -X 'GET' \
  'http://127.0.0.1:8000/books' \
  -H 'accept: application/json'
```

Request URL

http://127.0.0.1:8000/books

Server response

Code Details

200

Response body

```
{
  "id": 2,
  "title": "Advanced SQL",
  "author": "Josephine Ashley",
  "category": "Database",
  "available": true
},
{
  "id": 3,
  "title": "Data Science Intro",
  "author": "Rhinney Ashley",
  "category": "Data",
  "available": true
},
{
  "id": 4,
  "title": "Machine Learning",
  "author": "Thomas Ashley"
}
```

DELETE /books/{id}/ Delete Book

127.0.0.1:8000/docs#/default/delete_book_books_book_id_delete

DELETE /books/{book_id} Delete Book

Parameters

Cancel

Name	Description
book_id * required integer (path)	25

Execute

Responses

Code	Description	Links
200	Successful Response	No links

Media type
application/json

Controls Accept header.

Example Value | Schema

API Design for Library | Library API Assignment Solution | Limkokwing Library REST API

127.0.0.1:8000/docs#/default/delete_book_books_book_id_delete

Responses

Curl

```
curl -X 'DELETE' \
'http://127.0.0.1:8000/books/25' \
-H 'accept: application/json'
```

Request URL

http://127.0.0.1:8000/books/25

Server response

Code	Details
200	Response body <pre>{ "message": "Book deleted" }</pre> Response headers <pre>content-length: 26 content-type: application/json date: Wed, 29 Apr 2026 11:06:01 GMT server: uvicorn</pre>

Responses

Code	Description	Links
------	-------------	-------

GET /books after delete (DELETE BOOK)

The screenshot shows a REST client interface with the following details:

- URL:** 127.0.0.1:8000/docs#/default/get_books_books_get
- Method:** GET
- Path:** /books Get Books
- Parameters:**

Name	Description
title string (string null) (query)	title
author string (string null) (query)	author
category string (string null) (query)	category
- Buttons:** Execute, Clear, Cancel
- Responses:** (Empty section)

The screenshot shows the response of the GET /books endpoint. The response body is a JSON array of book objects:

```
curl -X 'GET' \
'http://127.0.0.1:8000/books' \
-H 'accept: application/json'
```

Request URL: http://127.0.0.1:8000/books

Server response: 200

Response body:

```
{
  "title": "Computer Graphics",
  "author": "Mohamed Yaffa",
  "category": "Graphics",
  "available": true
},
{
  "id": 24,
  "title": "Game Development",
  "author": "Amie Yaffa",
  "category": "Game Dev",
  "available": true
},
{
  "id": 26,
  "title": "Embedded Systems",
  "author": "Muhamadu Yaffa",
  "category": "Engineering",
  "available": true
}
```

Appendix E: Tools and Technologies Used

- Programming Language: Python
- Framework: FastAPI
- Server: Uvicorn
- Development Environment: Visual Studio Code
- API Testing: Swagger UI (/docs)
- Version Control: GitHub

Appendix F: GitHub Repository

Repository Name: library-api-fastapi-OOP-Method--2-Joshua-M-K-Yaffa--Semester-4---BSEM1201---Assignment-1

Repository Link: <https://github.com/JoshuaYaffa/JoshuaYaffa-library-api-fastapi-OOP-Method--2-Joshua-M-K-Yaffa--Semester-4---BSEM1201---Assignment-1.git>

API DESIGN (SYSTEM ARCHITECTURE DIAGRAM)

Appendix G: Assumptions

- Data is stored in memory (no database used)
- Users are predefined
- Fine calculation is based on number of overdue days
- No authentication is implemented